

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN



BÀI TẬP 3

Môn học	: Seminar chuyên đề	
Sinh viên thực hiện	: Nguyễn Hồng Thắm	3122411194
Lớp	: DCT122C3	
Giảng viên hướng dẫn	: TS. Đỗ Như Tài	

Khoa: Công Nghệ Thông Tin
Thành Phố Hồ Chí Minh, Năm 2026

TỔNG QUAN KIẾN TRÚC PHẦN MỀM

.....

TRẮC NGHIỆM KIẾN TRÚC

Kiến thức cơ bản về kiến trúc phần mềm

1. Kiến trúc nào dưới đây không thuộc các loại kiến trúc phần mềm phổ biến?
 - A. Monolithic Architecture
 - B. Layered Architecture
 - C. Modular Monolithic Architecture
 - D. Quantum Architecture
2. Nguyên tắc "YAGNI" đề cập đến điều gì?
 - A. Không thêm các tính năng mà bạn sẽ không cần đến
 - B. Thiết kế kiến trúc theo mô-đun nhỏ
 - C. Không phụ thuộc vào công nghệ cụ thể
 - D. Luôn áp dụng mô hình microservices
3. Khi nào nên sử dụng kiến trúc monolithic thay vì microservices?
 - A. Khi hệ thống cần khả năng mở rộng cao
 - B. Khi ứng dụng nhỏ và ít thay đổi
 - C. Khi cần xử lý hàng triệu yêu cầu mỗi giây
 - D. Khi có nhiều đội phát triển độc lập

Thiết kế và triển khai microservices

4. Trong kiến trúc microservices, mô hình "Database-per-Service Pattern" đề xuất:
 - A. Một cơ sở dữ liệu chung cho tất cả các dịch vụ
 - B. Mỗi dịch vụ có cơ sở dữ liệu riêng của mình
 - C. Chỉ sử dụng cơ sở dữ liệu NoSQL
 - D. Tích hợp dữ liệu bằng cách sử dụng API Gateway
5. Lợi ích chính của sử dụng gRPC trong microservices là gì?
 - A. Giao tiếp không đồng bộ
 - B. Hỗ trợ truyền dữ liệu nhanh với mã hóa nhị phân
 - C. Giao tiếp hai chiều thời gian thực
 - D. Đơn giản hóa thiết kế RESTful

6. Pattern nào dưới đây được sử dụng để xử lý truy vấn dữ liệu và lệnh ghi riêng biệt trong microservices?

- A. Event Sourcing
- B. CQRS (Command Query Responsibility Segregation)**
- C. Materialized View Pattern
- D. Database Sharding Pattern

Xử lý giao tiếp và dữ liệu

7. Pattern nào giúp giảm độ trễ trong giao tiếp giữa các microservices?

- A. Service Aggregator Pattern
- B. Publish/Subscribe Messaging**
- C. Backends for Frontends Pattern
- D. API Gateway Pattern

8. "Event Sourcing Pattern" hoạt động như thế nào?

- A. Lưu trữ trạng thái hiện tại của hệ thống trong cơ sở dữ liệu
- B. Lưu trữ tất cả các sự kiện thay đổi trạng thái hệ thống**
- C. Tạo các bản sao dữ liệu để truy vấn nhanh hơn
- D. Kết hợp nhiều dịch vụ thành một giao tiếp duy nhất

9. Trong mô hình giao dịch phân tán, SAGA Pattern sử dụng:

- A. Giao dịch nguyên tử trên tất cả các dịch vụ
- B. Các bước nhỏ độc lập với khả năng khôi phục khi thất bại**
- C. Một cơ sở dữ liệu duy nhất để đảm bảo tính nhất quán
- D. Kiểm tra tất cả dữ liệu trước khi ghi vào cơ sở dữ liệu

Resilience và triển khai hệ thống

10. Pattern nào không thuộc các chiến lược resilience phổ biến?

- A. Circuit Breaker
- B. Bulkhead
- C. Timeout
- D. Scale Cube**

11. Chiến lược triển khai nào đảm bảo không có thời gian chết khi cập nhật hệ thống?

- A. Blue-Green Deployment**
- B. Rolling Deployment
- C. Canary Deployment
- D. A/B Testing

12. Công cụ nào không được sử dụng để giám sát và theo dõi microservices?

- A. Prometheus
- B. Elasticsearch
- C. Kubernetes**

D. Grafana

Các bài toán thực tế

13. Khi nào nên sử dụng kiến trúc event-driven?

- A. Khi cần xử lý dữ liệu thời gian thực và khối lượng lớn
- B. Khi giao tiếp giữa các dịch vụ đơn giản
- C. Khi cần đảm bảo tính nhất quán mạnh mẽ
- D. Khi giao dịch chỉ liên quan đến một dịch vụ

14. Giải pháp nào phù hợp nhất để xử lý vấn đề bottleneck ở cơ sở dữ liệu trong microservices?

- A. Sử dụng REST API
- B. Triển khai caching phân tán
- C. Thêm nhiều dịch vụ vào hệ thống
- D. Chuyển toàn bộ cơ sở dữ liệu sang NoSQL

15. CQRS giúp ích gì trong kiến trúc microservices?

- A. Tăng tính đồng bộ của giao tiếp
- B. Tăng hiệu năng khi xử lý truy vấn và ghi dữ liệu
- C. Đảm bảo tính nhất quán dữ liệu mạnh mẽ
- D. Đơn giản hóa thiết kế giao tiếp giữa các dịch vụ

CÂU HỎI LÝ THUYẾT

Kiến thức cơ bản về kiến trúc phần mềm

1. Các kiến trúc phần mềm chính bao gồm những loại nào? So sánh ưu và nhược điểm của chúng.

a. Monolithic Architecture

Khái niệm:

- Monolithic Architecture là kiến trúc trong đó toàn bộ chức năng của ứng dụng được xây dựng và triển khai trong một khối thống nhất. Các thành phần như giao diện, xử lý nghiệp vụ và truy cập dữ liệu được gắn chặt với nhau.

Ưu điểm:

- Dễ thiết kế và triển khai.
- Phù hợp với ứng dụng nhỏ, ít người dùng.
- Quản lý đơn giản, ít công cụ phức tạp.

Nhược điểm:

- Khó mở rộng khi hệ thống lớn dần.

- Một lỗi nhỏ có thể làm sập toàn bộ hệ thống.
- Khó bảo trì và nâng cấp từng phần riêng lẻ.

b. Modular Monolithic Architecture

Khái niệm:

- Là dạng cải tiến của monolithic, trong đó ứng dụng vẫn triển khai như một khối duy nhất nhưng được chia thành các mô-đun rõ ràng theo chức năng.

Ưu điểm:

- Dễ tổ chức mã nguồn, tăng khả năng bảo trì.
- Giảm sự phụ thuộc giữa các thành phần.
- Phù hợp với hệ thống vừa và nhỏ.

Nhược điểm:

- Vẫn bị giới hạn về khả năng mở rộng độc lập.
- Khi triển khai vẫn phải triển khai toàn bộ hệ thống.

c. Microservices Architecture

Khái niệm:

- Microservices chia ứng dụng thành nhiều dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhiệm một chức năng riêng và có thể được phát triển, triển khai độc lập.

Ưu điểm:

- Khả năng mở rộng cao theo từng dịch vụ.
- Cách ly lỗi tốt, tăng độ ổn định hệ thống.
- Linh hoạt trong việc lựa chọn công nghệ.

Nhược điểm:

- Phức tạp trong thiết kế và vận hành.
- Tăng chi phí quản lý, giám sát.
- Khó đảm bảo tính nhất quán dữ liệu.

2. Nguyên tắc KISS, YAGNI, và Separation of Concerns (SoC) có vai trò gì trong thiết kế hệ thống phần mềm?

KISS (Keep It Simple, Stupid):

- Nhấn mạnh việc thiết kế hệ thống càng đơn giản càng tốt, tránh các giải pháp phức tạp không cần thiết. Điều này giúp hệ thống dễ hiểu, dễ bảo trì và giảm lỗi.

YAGNI (You Aren't Gonna Need It):

- Nguyên tắc này khuyến cáo không nên phát triển các tính năng chưa có nhu cầu thực tế, giúp tránh lãng phí thời gian và tài nguyên.

Separation of Concerns (SoC):

- Phân tách rõ ràng trách nhiệm giữa các thành phần trong hệ thống, giúp mã nguồn rõ ràng, dễ mở rộng và dễ thay đổi khi yêu cầu thay đổi.

3. Khi nào nên chọn kiến trúc monolithic và khi nào nên chuyển sang microservices?

Nên chọn Monolithic khi:

- Ứng dụng nhỏ hoặc mới bắt đầu.
- Ít thay đổi yêu cầu.
- Đội phát triển nhỏ, nguồn lực hạn chế.

Nên chuyển sang Microservices khi:

- Hệ thống lớn, lượng người dùng tăng nhanh.
- Cần mở rộng linh hoạt theo từng chức năng.
- Có nhiều nhóm phát triển làm việc song song.

Thiết kế và triển khai microservices

4. Mô hình Database-per-Service Pattern là gì? Ưu và nhược điểm của nó trong microservices?

Khái niệm:

- Database-per-Service Pattern đề xuất rằng mỗi microservice sở hữu một cơ sở dữ liệu riêng, không chia sẻ trực tiếp với dịch vụ khác.

Ưu điểm:

- Đảm bảo tính độc lập và cách ly dữ liệu.
- Dễ mở rộng và thay đổi cấu trúc dữ liệu.
- Giảm phụ thuộc giữa các dịch vụ.

Nhược điểm:

- Khó đảm bảo tính nhất quán dữ liệu.
- Phức tạp trong quản lý giao dịch phân tán.
- Tăng chi phí vận hành.

5. Các cách tiếp cận phân mảnh dữ liệu (Data Partitioning) gồm những loại nào?

Horizontal Partitioning:

Phân chia dữ liệu theo hàng, giúp xử lý dữ liệu lớn hiệu quả.

Vertical Partitioning:

Phân chia theo cột, tối ưu truy vấn cho từng nhóm dữ liệu.

Functional Partitioning:

Phân chia dữ liệu theo từng chức năng nghiệp vụ.

6. Hãy giải thích các mô hình giao tiếp giữa các microservices như RESTful API, gRPC, và WebSocket. Ưu và nhược điểm của từng mô hình.

- Trong kiến trúc microservices, các dịch vụ hoạt động độc lập và cần giao tiếp với nhau thông qua mạng. Việc lựa chọn mô hình giao tiếp phù hợp ảnh hưởng trực tiếp đến hiệu năng, khả năng mở rộng và độ phức tạp của hệ thống.

- Ba mô hình giao tiếp phổ biến là RESTful API, gRPC và WebSocket.

1. RESTful API

Khái niệm

- RESTful API là mô hình giao tiếp dựa trên giao thức HTTP, sử dụng các phương thức quen thuộc như GET, POST, PUT, DELETE để trao đổi dữ liệu, thường ở định dạng JSON.

Ưu điểm

- Đơn giản, dễ hiểu và dễ triển khai.
- Phổ biến, được hỗ trợ rộng rãi bởi hầu hết các ngôn ngữ và framework
- Phù hợp cho giao tiếp giữa client và server hoặc các microservices quy mô nhỏ.

Nhược điểm

- Hiệu suất không cao do dữ liệu dạng text (JSON).
- Không tối ưu cho các hệ thống có tần suất gọi lớn.
- Giao tiếp chủ yếu theo mô hình request–response, ít linh hoạt.

2. gRPC

Khái niệm

- gRPC là framework giao tiếp hiệu năng cao, sử dụng HTTP/2 và Protocol Buffers (mã hóa nhị phân) để truyền dữ liệu giữa các microservices.

Ưu điểm

- Hiệu suất cao, tốc độ truyền nhanh.
- Hỗ trợ streaming hai chiều.
- Phù hợp với hệ thống microservices lớn, yêu cầu tải cao.

Nhược điểm

- Khó debug và kiểm thử hơn REST.

- Yêu cầu định nghĩa interface (proto file).
- Ít thân thiện với trình duyệt so với REST.

3. WebSocket

Khái niệm

- WebSocket là mô hình giao tiếp hai chiều, thời gian thực, duy trì kết nối lâu dài giữa client và server.

Ưu điểm

- Giao tiếp real-time, độ trễ thấp.
- Phù hợp với ứng dụng chat, notification, dashboard thời gian thực.
- Giảm chi phí thiết lập kết nối liên tục.

Nhược điểm

- Quản lý kết nối phức tạp.
- Khó mở rộng với số lượng kết nối lớn.
- Không phù hợp cho giao tiếp request–response truyền thống.

Xử lý dữ liệu và giao dịch

7. CQRS là gì? Khi nào nên áp dụng mô hình này?

- CQRS (Command Query Responsibility Segregation) là mô hình tách riêng xử lý ghi dữ liệu (command) và truy vấn dữ liệu (query).

Nên áp dụng khi:

- Hệ thống có nhiều truy vấn đọc.
- Cần tối ưu hiệu năng và khả năng mở rộng.
- Có yêu cầu cao về hiệu suất.

8. Hãy so sánh SAGA Pattern với các giao dịch truyền thống trong quản lý giao dịch phân tán.

Giao dịch truyền thống:

- Đảm bảo tính nguyên tử (ACID) nhưng khó mở rộng trong môi trường phân tán.

SAGA Pattern:

- Chia giao dịch lớn thành nhiều bước nhỏ, mỗi bước có cơ chế bù trừ khi lỗi xảy ra, phù hợp với microservices.

9. Sự khác biệt giữa Event Sourcing Pattern và Transactional Outbox Pattern là gì? Resilience và triển khai hệ thống

Event Sourcing Pattern:

- Lưu trữ toàn bộ các sự kiện thay đổi trạng thái hệ thống, hỗ trợ phục hồi

và audit.

Transactional Outbox Pattern:

- Đảm bảo dữ liệu được ghi và phát sự kiện một cách nhất quán, tránh mất dữ liệu.

10. Mô hình triển khai nào được sử dụng để giảm thiểu thời gian chết khi triển khai microservices? (VD: blue-green, rolling, canary)

1. Blue-Green Deployment

Khái niệm:

Blue-Green Deployment sử dụng hai môi trường triển khai song song:

- Blue: phiên bản hệ thống đang hoạt động (production hiện tại).
- Green: phiên bản hệ thống mới được triển khai.

Khi phiên bản Green được kiểm tra ổn định, hệ thống sẽ chuyển toàn bộ lưu lượng người dùng từ Blue sang Green.

2. Rolling Deployment

Khái niệm:

- Rolling Deployment cập nhật hệ thống từng phần một, ví dụ cập nhật từng instance hoặc từng pod thay vì toàn bộ hệ thống cùng lúc.

3. Canary Deployment

Khái niệm:

- Canary Deployment triển khai phiên bản mới cho một phần nhỏ người dùng trước (ví dụ 5–10%). Nếu hệ thống hoạt động ổn định, phiên bản mới sẽ dần dần được mở rộng cho toàn bộ người dùng.

11. Các resilience patterns phổ biến trong microservices bao gồm những gì? Mỗi mô hình giải quyết vấn đề gì?

1. Retry Pattern

Mục đích:

- Xử lý các lỗi tạm thời xảy ra trong quá trình giao tiếp giữa các microservices.

Cách hoạt động:

- Khi một yêu cầu thất bại (do timeout, lỗi mạng ngắn hạn...), hệ thống sẽ thử gửi lại yêu cầu sau một khoảng thời gian nhất định.

Vấn đề được giải quyết:

- Lỗi mạng tạm thời.
- Service bị quá tải trong thời gian ngắn.

2. Circuit Breaker Pattern

Mục đích:

- Ngăn chặn lỗi lan truyền khi một dịch vụ đang gặp sự cố nghiêm trọng.

Cách hoạt động:

Circuit Breaker theo dõi số lần lỗi:

- Nếu lỗi vượt ngưỡng → ngắt kết nối tạm thời (open state).
- Không gửi thêm request đến service đang lỗi.
- Sau một thời gian → thử kết nối lại (half-open).

Vấn đề được giải quyết:

- Tránh làm quá tải service đang bị lỗi.
- Ngăn hiệu ứng domino trong hệ thống phân tán.

3. Bulkhead Pattern

Mục đích:

- Cách ly tài nguyên để lỗi ở một phần không ảnh hưởng đến toàn hệ thống.

Cách hoạt động:

Chia hệ thống thành các “ngăn” độc lập (giống khoang tàu thủy):

- Mỗi service hoặc nhóm service dùng tài nguyên riêng (thread, connection, memory).
- Khi một service bị lỗi, các service khác vẫn hoạt động bình thường.

Vấn đề được giải quyết:

- Lỗi hoặc quá tải ở một dịch vụ kéo sập toàn bộ hệ thống.
- Sử dụng tài nguyên không kiểm soát.

4. Timeout Pattern

Mục đích:

- Tránh việc chờ đợi vô hạn khi một service phản hồi chậm hoặc không phản hồi.

Cách hoạt động:

- Mỗi request được cấu hình một thời gian chờ tối đa. Nếu quá thời gian này, request sẽ bị hủy và xử lý theo logic dự phòng.

Vấn đề được giải quyết:

- Service phản hồi chậm.

- Tắc nghẽn tài nguyên do request treo lâu.

12. Làm thế nào để giám sát và quan sát microservices sử dụng các công cụ như Prometheus, Grafana, hoặc Elastic Stack?

1. Giám sát microservices với Prometheus

Vai trò:

- Prometheus là công cụ thu thập và lưu trữ các chỉ số (metrics) của hệ thống microservices.

Cách hoạt động:

- Mỗi microservice xuất ra các metrics (CPU, RAM, số request, latency, tỉ lệ lỗi...).
- Prometheus định kỳ pull dữ liệu từ các service hoặc exporter.
- Dữ liệu được lưu dưới dạng time-series.

Lợi ích:

- Theo dõi hiệu năng hệ thống theo thời gian thực.
- Phát hiện sớm các dấu hiệu quá tải hoặc suy giảm hiệu năng.
- Hỗ trợ thiết lập cảnh báo (alert) khi vượt ngưỡng cho phép.

2. Trực quan hóa dữ liệu giám sát với Grafana

Vai trò:

- Grafana là công cụ trực quan hóa dữ liệu giám sát, thường được sử dụng kết hợp với Prometheus.

Cách hoạt động:

- Kết nối với Prometheus làm nguồn dữ liệu.
- Hiển thị metrics dưới dạng biểu đồ, dashboard trực quan.

Lợi ích:

- Giúp đội vận hành dễ dàng theo dõi tình trạng hệ thống.
- Phân tích xu hướng tăng giảm hiệu năng.
- Hỗ trợ ra quyết định nhanh khi có sự cố xảy ra.

3. Quan sát và phân tích log với Elastic Stack

Elastic Stack gồm:

- Elasticsearch: lưu trữ và tìm kiếm log.

- Logstash / Beats: thu thập và xử lý log.
- Kibana: trực quan hóa và phân tích log.

Cách hoạt động:

- Các microservices ghi log (request, error, event...).
- Log được gửi về Elasticsearch thông qua Logstash hoặc Beats.
- Kibana hiển thị log, hỗ trợ tìm kiếm và phân tích.

Lợi ích:

- Theo dõi chi tiết hành vi của từng service.
- Phân tích nguyên nhân lỗi (debug).
- Hỗ trợ điều tra sự cố trong hệ thống phân tán.

4. Kết hợp các công cụ để quan sát toàn diện

Việc giám sát microservices hiệu quả cần kết hợp cả metrics và logs:

- Prometheus: biết hệ thống có vấn đề hay không
- Grafana: biết vấn đề xảy ra khi nào và mức độ ra sao
- Elastic Stack: biết vì sao vấn đề xảy ra

Sự kết hợp này giúp hệ thống đạt được observability toàn diện, từ hiệu năng, trạng thái đến nguyên nhân lỗi.

Các bài toán thực tế

12. Giải pháp nào giúp tăng hiệu năng khi giao tiếp giữa các microservices? (VD: cache phân tán, giảm số lượng API call)

- Trong kiến trúc microservices, việc giao tiếp giữa các dịch vụ qua mạng có thể gây độ trễ cao và tăng tải hệ thống. Để cải thiện hiệu năng, có thể áp dụng các giải pháp sau:

a. Sử dụng cache phân tán

- Lưu trữ các dữ liệu được truy cập thường xuyên vào bộ nhớ đệm (cache).
- Giảm số lần truy cập trực tiếp vào cơ sở dữ liệu hoặc gọi sang service khác.
- Giúp giảm độ trễ và tăng tốc độ phản hồi.

b. Giảm số lượng API call giữa các services

- Tránh việc một request phải gọi quá nhiều service con.
- Áp dụng các pattern như Service Aggregator hoặc API Gateway Aggregation để tổng hợp dữ liệu.

- Giảm chi phí mạng và cải thiện hiệu suất tổng thể.

c. Sử dụng giao tiếp bất đồng bộ

- Thay vì gọi đồng bộ (request-response), các service có thể giao tiếp thông qua message hoặc event.
- Giúp hệ thống không bị block khi chờ phản hồi.
- Tăng khả năng chịu tải và mở rộng.

d. Tối ưu giao thức giao tiếp

- Sử dụng gRPC thay cho REST trong các trường hợp cần hiệu suất cao.
- Giảm kích thước dữ liệu truyền và thời gian xử lý.

Kết luận:

- Việc kết hợp cache phân tán, giảm API call và giao tiếp bất đồng bộ giúp cải thiện đáng kể hiệu năng khi giao tiếp giữa các microservices.

13. Làm thế nào để xử lý bài toán hiệu suất cho ứng dụng cần xử lý hàng triệu sự kiện mỗi giây?

- Với các hệ thống có yêu cầu xử lý khối lượng sự kiện cực lớn, cần áp dụng các giải pháp kiến trúc phù hợp để đảm bảo hiệu suất và khả năng mở rộng.

a. Áp dụng kiến trúc Event-driven

- Các sự kiện được phát sinh và xử lý độc lập.
- Các service không phụ thuộc chặt chẽ vào nhau.
- Giảm tải cho các service xử lý đồng bộ.

b. Sử dụng stream processing

- Xử lý luồng dữ liệu liên tục theo thời gian thực.
- Phù hợp với các hệ thống log, dữ liệu IoT, giao dịch lớn.
- Giúp xử lý nhanh và song song lượng sự kiện lớn.

c. Mở rộng theo chiều ngang (horizontal scaling)

- Nhân bản nhiều instance của service xử lý sự kiện.
- Phân phối tải đều để tránh bottleneck.

d. Phân mảnh dữ liệu và tải

- Chia dữ liệu theo key, theo chức năng hoặc theo luồng xử lý.
- Giảm áp lực lên một thành phần duy nhất.

Kết luận:

- Để xử lý hàng triệu sự kiện mỗi giây, cần kết hợp event-driven

architecture, stream processing và khả năng mở rộng ngang, giúp hệ thống hoạt động ổn định và hiệu quả dưới tải lớn.

14. Khi nào cần sử dụng kiến trúc event-driven thay vì request-response?

- Kiến trúc request-response phù hợp với các hệ thống đơn giản, nhưng trong nhiều trường hợp, event-driven architecture mang lại hiệu quả cao hơn.

Nên sử dụng event-driven khi:

- Hệ thống cần xử lý bất đồng bộ, không cần phản hồi ngay lập tức.
- Khối lượng yêu cầu lớn, cần giảm độ trễ và tránh tắc nghẽn.
- Các service cần tách biệt và ít phụ thuộc vào nhau.
- Ứng dụng yêu cầu xử lý dữ liệu thời gian thực.
- Cần khả năng mở rộng linh hoạt theo tải.

Tư duy thiết kế

16. Tại sao "mỗi quyết định thiết kế cần được biện minh bởi một yêu cầu kinh doanh" là quan trọng?

- Trong thiết kế phần mềm, kiến trúc và công nghệ không phải mục tiêu cuối cùng, mà chỉ là phương tiện để phục vụ mục tiêu kinh doanh. Vì vậy, mỗi quyết định thiết kế cần được biện minh rõ ràng bằng một yêu cầu kinh doanh cụ thể.

Ý nghĩa của nguyên tắc này:

a. Đảm bảo hệ thống tạo ra giá trị thực tế

- Các quyết định thiết kế dựa trên yêu cầu kinh doanh giúp hệ thống giải quyết đúng vấn đề cần giải quyết.
- Tránh xây dựng các tính năng hoặc kiến trúc không mang lại lợi ích rõ ràng cho người dùng hoặc doanh nghiệp.

b. Tránh lãng phí tài nguyên

- Thiết kế quá phức tạp sẽ làm tăng chi phí phát triển, vận hành và bảo trì.
- Khi mỗi quyết định đều gắn với mục tiêu kinh doanh, tài nguyên (thời gian, nhân lực, chi phí) được sử dụng hiệu quả hơn.

c. Giúp cân bằng giữa kỹ thuật và mục tiêu dài hạn

- Yêu cầu kinh doanh định hướng mức độ mở rộng, hiệu năng và độ ổn định của hệ thống.
- Tránh áp dụng công nghệ mới chỉ vì “xu hướng” mà không phù hợp với quy mô hệ thống.

d. Hỗ trợ ra quyết định và đánh giá thiết kế

- Khi yêu cầu thay đổi, đội ngũ dễ dàng đánh giá lại các quyết định thiết kế.
- Giúp kiến trúc linh hoạt và dễ thích nghi với sự thay đổi của thị trường.

17. Hãy giải thích khái niệm "Over-engineering" trong thiết kế phần mềm. Làm thế nào để tránh?

a. Khái niệm Over-engineering

Over-engineering là tình trạng thiết kế hệ thống quá phức tạp so với nhu cầu thực tế, bao gồm:

- Áp dụng kiến trúc hoặc công nghệ không cần thiết.
- Xây dựng quá nhiều tính năng chưa có yêu cầu sử dụng.
- Thiết kế cho các kịch bản “có thể xảy ra trong tương lai” nhưng chưa có cơ sở rõ ràng.

Hậu quả của over-engineering:

- Tăng chi phí phát triển và vận hành.
- Hệ thống khó hiểu, khó bảo trì.
- Giảm tốc độ phát triển và khả năng thích nghi.

b. Cách tránh Over-engineering

1. Áp dụng các nguyên tắc thiết kế

- KISS: giữ thiết kế đơn giản nhất có thể.
- YAGNI: không phát triển tính năng chưa cần thiết.
- Separation of Concerns: phân tách rõ ràng trách nhiệm.

2. Dựa vào yêu cầu kinh doanh và thực tế sử dụng

- Chỉ thiết kế những gì phục vụ trực tiếp mục tiêu kinh doanh.
- Ưu tiên yêu cầu hiện tại, không phỏng đoán quá xa tương lai.

3. Thiết kế linh hoạt, mở rộng khi cần

- Xây dựng nền tảng đủ tốt để mở rộng sau này.
- Tránh tối ưu hóa quá sớm khi chưa có dữ liệu thực tế.

4. Đánh giá chi phí – lợi ích của mỗi quyết định

- Mỗi công nghệ hoặc mô hình mới cần mang lại giá trị rõ ràng.
- Nếu chi phí lớn hơn lợi ích → không nên áp dụng.